

데이터 지향 설계와 명시적 GPU 제어를 위한 멀티플랫폼 C++ 프레임워크 dy_framework

팀 명	DyD	학 과	컴퓨터소프트웨어공학과
팀 원	한재승(팀장), 윤훈, 정현진, 정준혁		
지도교수	정석용		

1. 작품의 개요 - 개발 배경과 필요성

GPU의 성능을 활용하는 출발점, 데이터와 실행 흐름의 설계.

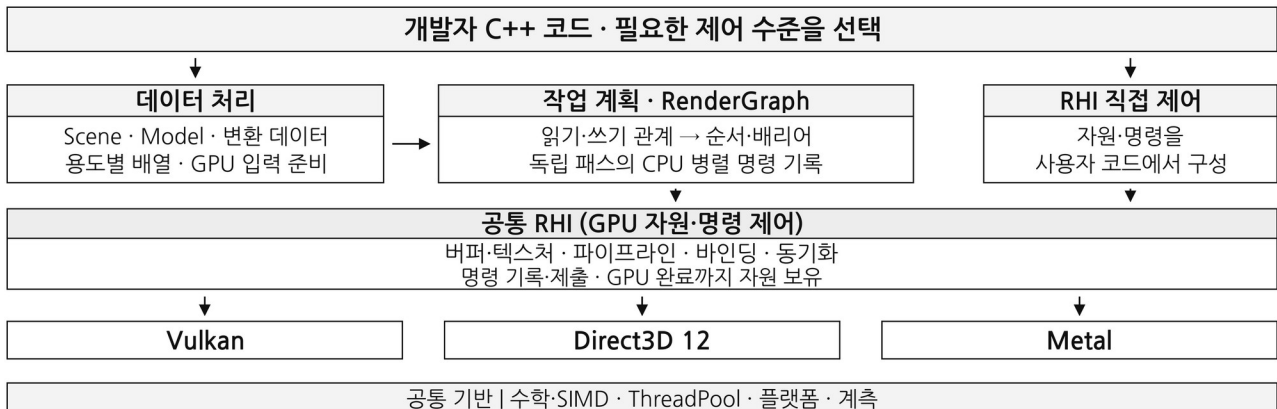
GPU(그래픽 처리 장치)가 그래픽스를 넘어 인공지능의 학습, 추론과 과학 시뮬레이션까지 활용 범위를 넓혀감에 따라 소프트웨어에도 CPU 캐시나 GPU 병렬 처리에 맞춘 하드웨어 친화적 설계가 중요해졌다. 기존 객체들의 기능, 관계로 정의되는 설계에서, 데이터의 배치-접근-변환을 우선하는 데이터 지향 설계(DOD)가 강조된다.^[1-3]

GPU의 본래 역할인 그래픽스 분야에서도 대량의 데이터를 어떤 단위로 모으고, 언제 읽고 쓸지 함께 설계하는 방향으로 패러다임 전환이 일어났다. Vulkan - Direct3D12 - Metal에서는 GPU 자원과 파이프라인, 명령 제출을 개발자가 구성하여 데이터 흐름과 실행 의존성을 직접 설계한다.^[4,5,7]

dy_framework는 이 설계 관점을 여러 플랫폼에서 구현하기 위한 C++ 개발 기반이다. 데이터 준비부터 GPU 자원과 명령 제어까지 연결하고, Vulkan·Direct3D 12·Metal의 차이를 공통 RHI로 다뤄 API 별 구현 부담을 줄인다.

2. 작품의 구성

데이터 처리, 작업 계획, 공통 RHI와 플랫폼별 구현으로 역할을 나눴다. RenderGraph는 패스의 의존성과 기록 순서를 구성하고, RHI는 명령 제출과 GPU 완료까지의 자원 보유를 맡는다.

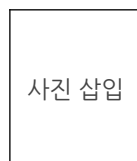


구성도 | 상위 데이터 경로와 RHI 직접 제어를 제공하며, 공통 기반이 각 계층을 지원한다.

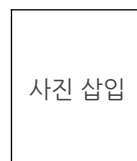
3. 프로젝트 개발환경 및 개발자

개발도구 및 사용 언어

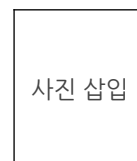
C++17, CMake
MSVC / GCC / Clang
HLSL, GLSL, Metal
Windows, Linux, macOS



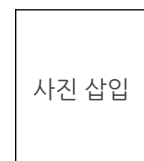
한재승
Core · RHI



윤훈
Vulkan



정현진
Direct3D 12



정준혁
Metal

4. 핵심기술 설명

명시적 GPU 제어부터 렌더링 최적화까지

RHI 위에 GPU 실행 제어와 고급 렌더링을 구성한다. 구현·개발 예정 기술을 독립적인 예제로 제공한다.

GPU 실행·데이터 제어	조명·영상 처리
Graphics / Compute Pipeline 셰이더·GPU 상태·바인딩을 실행 전에 구성한다.	Reverse-Z 깊이 범위를 반전해 깊이 버퍼의 정밀도를 활용한다.
Resource Barriers 자원 상태 전환과 패스 간 메모리 의존성을 지정한다.	Depth Sampling 깊이 텍스처를 후속 패스의 입력으로 읽어 처리한다.
Multithread Recording 독립 명령 목록을 CPU의 여러 스레드에서 병렬 기록한다.	Stencil Mask 스텐실 값으로 픽셀 처리 영역을 제한한다.
RenderGraph 읽기·쓰기 관계에서 작업 순서와 배리어를 계산한다.	PBR Lighting 금속성·거칠기 등 재질 속성으로 물리 기반 조명을 계산한다.
Frames / Present / Resource Reuse 프레임 제출·화면 표시와 GPU 자원 갱신·재사용을 제어한다.	Shadow Mapping 광원 시점의 깊이 정보로 그림자를 구성한다.
Instancing & Batching 호환되는 객체를 묶어 반복되는 그리기 작업을 줄인다.	Clustered Lighting 공간을 클러스터로 나눠 광원 목록을 셰이더로 전달한다.
Material & Texture Arrays 재질 바인딩과 인덱스 기반 텍스처 배열 참조를 구성한다.	Deferred Lighting G-buffer에 기록한 표면 정보로 조명을 계산한다.
CPU / GPU Profiling CPU 구간과 GPU 타임스탬프로 실행 비용을 측정한다.	HDR / Bloom / Tone Mapping 고명도 영상·밝은 영역 번짐·화면 출력 변환을 조합한다.
Data-Oriented Math / SIMD 행렬 배열을 순차 처리하고 SSE·NEON으로 행렬 곱을 계산한다.	Temporal Anti-Aliasing 이전 프레임 정보를 누적해 영상의 계단 현상을 줄인다.
Bindless Materials [예정] 셰이더가 인덱스로 다수의 재질·텍스처를 직접 참조한다.	Volumetric Fog 시선 경로를 따라 광원을 적분해 공간의 깊이를 표현한다.
Indirect Drawing / GPU Culling [예정] GPU가 가시성을 판정하고 그리기 인자를 생성한다.	BLAS / TLAS · Ray Query [예정] 장면 가속 구조와 광선 질의로 그림자를 계산한다.
Async Compute [예정] 그래픽스와 컴퓨트 작업을 별도 큐에 배치해 병렬 실행한다.	Ray-Traced Reflections [예정] 광선과 장면의 교차 결과로 반사 영상을 구성한다.
Mesh Shader [예정] 작업 그룹이 정점과 삼각형을 함께 생성·처리한다.	Ray Tracing Denoising [예정] 시간·공간 필터로 저샘플 광선 추적 영상의 잡음을 줄인다.
Meshlet Culling / LOD [예정] 메시를 작은 묶음으로 나눠 가시성과 상세도를 선택한다.	Hybrid Rendering [예정] 래스터화와 광선 추적 결과를 결합해 장면을 구성한다.

주요 이론서: **Real-Time Rendering, 4th Edition**^[6]

파이프라인·셰이딩·영상 처리·최적화의 관점을 참고하여, 실제 GPU 자원과 실행 명령으로 각 기법을 구성했다.

참고 자료 · 설계 관점과 렌더링 이론

- [1] Mike Acton (CppCon 2014). Data-Oriented Design and C++.
- [2] Stoyan Nikolov (CppCon 2018). OOP Is Dead, Long Live Data-oriented Design.
- [3] Timur Doumler (CppCon 2016). Want fast C++? Know your hardware!
- [4] Microsoft. Important Changes from Direct3D 11 to Direct3D 12.
- [5] Khronos. Vulkan Specification — Synchronization and Cache Control.
- [6] Akenine-Möller et al. Real-Time Rendering, 4th ed. A K Peters/CRC Press, 2018.
- [7] Apple. Metal — MTLHazardTrackingMode and resource synchronization.